

HW6 report

楊文德

寫法介紹

1. 遞迴窮舉法

```
int LCS_Exhaustive(const vector<double> &X, const vector<double> &Y, const vector<double> &Z, int i, int j, int k)
{
    if (i == 0 || j == 0 || k == 0)
        return 0;
    if (X[i - 1] == Y[j - 1] && X[i - 1] == Z[k - 1])
        return 1 + LCS_Exhaustive(X, Y, Z, i - 1, j - 1, k - 1);
    else
        return max({LCS_Exhaustive(X, Y, Z, i - 1, j, k),
                    LCS_Exhaustive(X, Y, Z, i, j - 1, k),
                    LCS_Exhaustive(X, Y, Z, i, j, k - 1)});
}

void LCS_Exhaustive_Print(const vector<double> &X, const vector<double> &Y, const vector<double> &Z,
                          int i, int j, int k, vector<double> &lcs, vector<double> &best)
{
    if (i == 0 || j == 0 || k == 0)
    {
        if (lcs.size() > best.size())
            best = lcs;
        return;
    }
    if (X[i - 1] == Y[j - 1] && X[i - 1] == Z[k - 1])
    {
        lcs.push_back(X[i - 1]);
        LCS_Exhaustive_Print(X, Y, Z, i - 1, j - 1, k - 1, lcs, best);
        lcs.pop_back();
    }
    else
    {
        LCS_Exhaustive_Print(X, Y, Z, i - 1, j, k, lcs, best);
        LCS_Exhaustive_Print(X, Y, Z, i, j - 1, k, lcs, best);
        LCS_Exhaustive_Print(X, Y, Z, i, j, k - 1, lcs, best);
    }
}
```

我們透過以下的遞迴方程式實做出 recursive 的程式，可以在 LCS_Exhaustive 函數中看到方程的影子：

$$C[i, j, z] = \begin{cases} C[i-1, j-1, z-1] + 1 & \text{if } x = y = z \\ \max\{C[i-1, j, z], C[i, j-1, z], C[i, j, z-1]\} & \text{else} \end{cases}$$

當遞迴方程計算完後會使用 LCS_Exhaustive_Print 函數反向輸出先前遞回函數所建立起的路徑。

- 時間複雜度：我們可以透過觀察發現 LCS_Exhaustive 函數是三層遞迴，每層「至多」只會呼叫三次自己，因此時間複雜度可以寫成

$$O(3^{n+m+k}) = O(3^{\max(n,m,k)})$$

n,m,k 分別是 v1、v2、v3 的長度

※之所以能用 max 代替是因為 recursive tree 是由最多呼叫的遞迴所控制的

2. 動態規劃法

```
void LongestLCS(const vector<double> &v1, const vector<double> &v2, const vector<double> &v3, vector<vector<vector<int>>> &c, vector<vector<vector<tuple<int, int, int>>> &b)
{
    int n = v1.size();
    int m = v2.size();
    int f = v3.size();

    for (int i = 0; i <= n; i++)
        for (int j = 0; j <= m; j++)
            for (int k = 0; k <= f; k++)
                c[i][j][k] = 0;

    for (int i = 1; i <= n + 1; i++)
    {
        for (int j = 1; j <= m + 1; j++)
        {
            for (int z = 1; z <= f + 1; z++)
            {
                if (v1[i - 1] == v2[j - 1] && v3[z - 1] == v1[i - 1])
                {
                    c[i][j][z] = c[i - 1][j - 1][z - 1] + 1;
                    b[i][j][z] = {-1, -1, -1};
                }
                else if (c[i - 1][j][z] >= c[i][j - 1][z] && c[i - 1][j][z] >= c[i][j][z - 1])
                {
                    c[i][j][z] = c[i - 1][j][z];
                    b[i][j][z] = {-1, 0, 0};
                }
                else if (c[i][j - 1][z] >= c[i - 1][j][z] && c[i][j - 1][z] >= c[i][j][z - 1])
                {
                    c[i][j][z] = c[i][j - 1][z];
                    b[i][j][z] = {0, -1, 0};
                }
                else
                {
                    c[i][j][z] = c[i][j][z - 1];
                    b[i][j][z] = {0, 0, -1};
                }
            }
        }
    }
}
```

```
void Print_LCS(const vector<vector<vector<tuple<int, int, int>>> &b, const vector<double> &v, int i, int j, int z)
{
    if (i == 0 || j == 0 || z == 0)
        return;

    if (b[i][j][z] == make_tuple(-1, -1, -1))
    {
        Print_LCS(b, v, i - 1, j - 1, z - 1);
        cout << v[i - 1] << " ";
    }
    else if (b[i][j][z] == make_tuple(-1, 0, 0))
    {
        Print_LCS(b, v, i - 1, j, z);
    }
    else if (b[i][j][z] == make_tuple(0, -1, 0))
    {
        Print_LCS(b, v, i, j - 1, z);
    }
    else
    {
        Print_LCS(b, v, i, j, z - 1);
    }
}
```

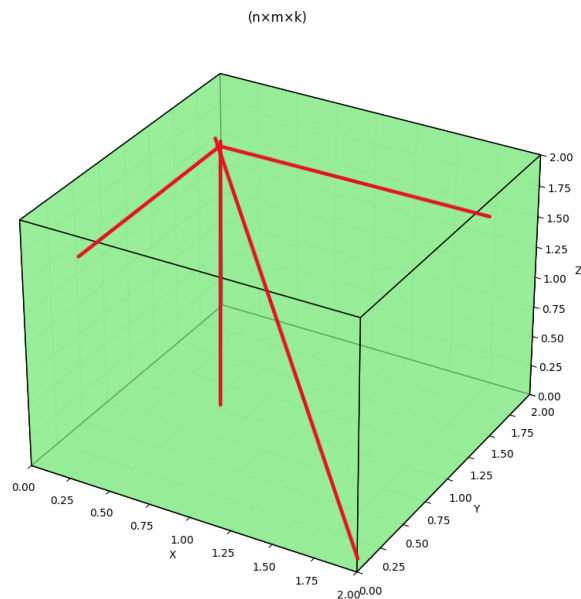
這部分寫法我是參照老師在講義上所提供的 pseudocode 所啟發的，那時候我陷入兩種選擇：

1. 用老師在講義上所提供找兩組陣列的 LCS，雖然不能直接用在三組上，但我覺得先用在前三組產生一組 LCS 後再作用在第三組陣列上說不定是可行的，但結論是我會忽略元素出現的時間，這樣可能造成結果存在著誤差。
2. 依照座標系的關係，根據兩組是二維座標推斷三組應該要是三維座標系，知道這消息後我就開始著手設計演算法的部分

$$C[i, j, z] = \begin{cases} C[i-1, j-1, z-1] + 1 & \text{if } x = y = z \\ \max\{C[i-1, j, z], C[i, j-1, z], C[i, j, z-1]\} & \text{else} \end{cases}$$

透過在遞迴窮舉法中列出新設計的遞迴方程，我們可以清楚看到多了一個變數控制高度的部分，這樣原本 **b** 矩陣返回時座標也不能想之前那樣只有 3 個向量，因為多了一個變數所以改成四個向量：

$$(-1, -1), (-1, 0), (0, -1) \rightarrow (-1, -1, -1), (-1, 0, 0), (0, -1, 0), (0, 0, -1)$$



▲示意圖(紅線為向量軌跡)

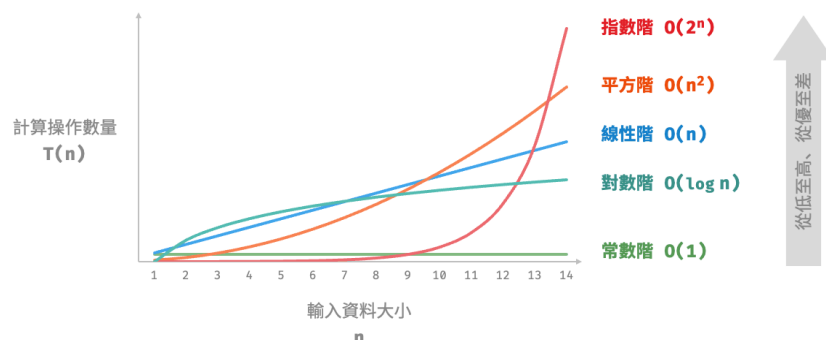
新設計的座標從 2 變數新增到 3 變數，因此儲存變數從 pair 變成 tuple

- 時間複雜度: DP 時做只用到三層 for loop，因此複雜度(空間也是)可以寫成

$$O(n \times m \times k)$$

n, m, k 分別是 $v1$ 、 $v2$ 、 $v3$ 的長度

透過比較遞迴與動態規劃兩種方法我們可以發現一個是指數增長，另一個是多項式增長，透過比較我們可以粗略地認知到 DP 遠比 recursive 來的快很多



心得

在這次作業中我原本很不了結老師上課所講/講義上所寫的，上網查之後才能順利理解，**DP** 真的是個很奇妙的東西，沒想到建個多維度陣列儲存資料就能夠存入狀態，並且在用向量陣列儲存回去的方向真的太酷了。